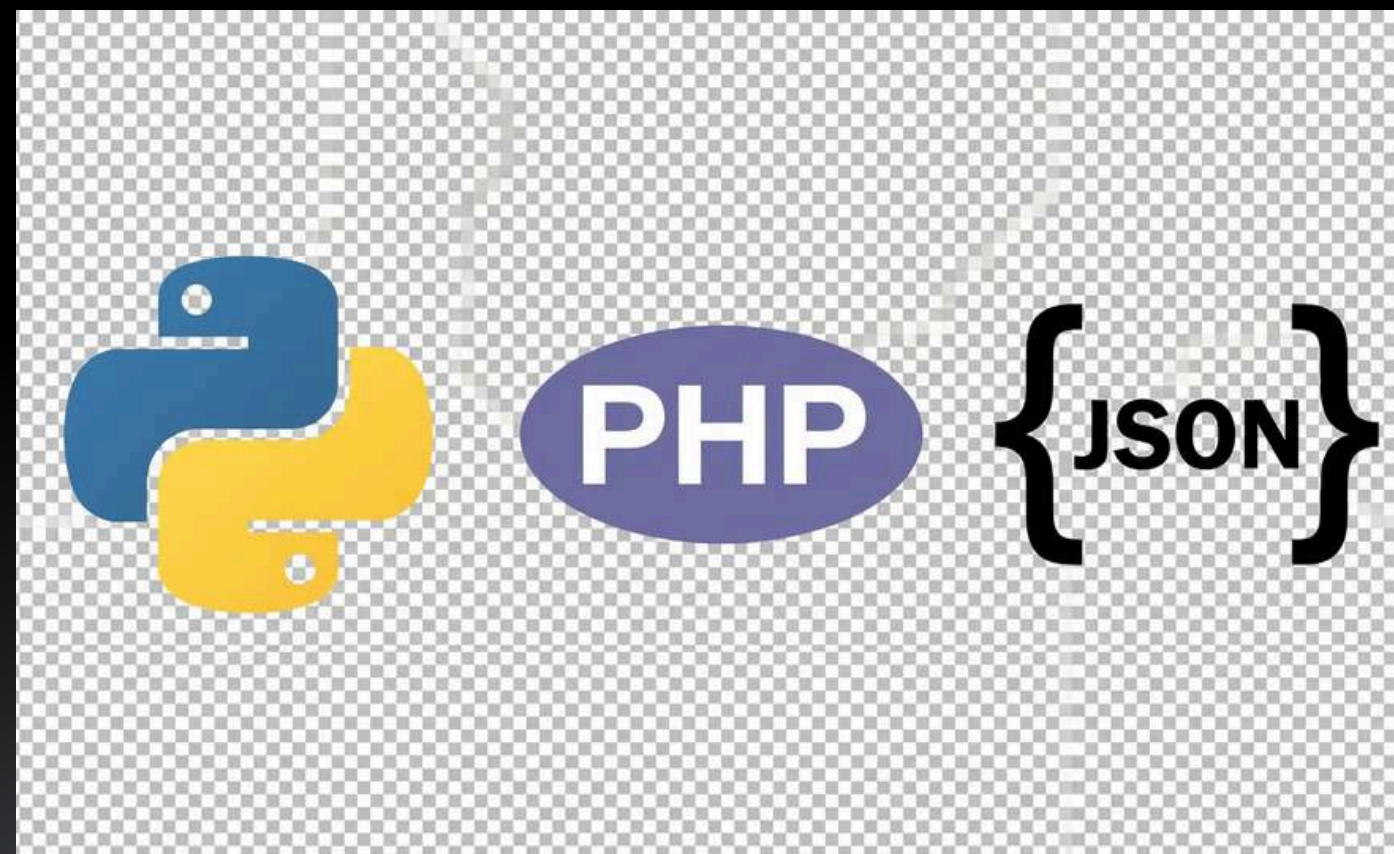


SISTEMA DE CONTROL DE ACCESO ARQUITECTURA JSON – PYTHON – PHP

INTEGRANTES:

- JAHAZIEL ISAI FRAIRE OLIVARES
- JESUS EMANUEL ARREOLA LOPEZ
- ALEXANDER ADONAI MOLINA RODRIGUEZ



EL PROBLEMA

**ESCENARIO:
ALMACÉN DE PRODUCTOS QUÍMICOS PELIGROSOS**

**PROBLEMA:
SE REQUIERE CONTROLAR QUIÉN PUEDE INGRESAR
PARA EVITAR RIESGOS Y ACCESOS NO AUTORIZADOS**

NECESIDAD:

- **SEGURIDAD DEL PERSONAL**
- **CONTROL DE SUSTANCIAS PELIGROSAS**
- **REGISTRO DE TODOS LOS ACCESOS**



¿POR QUÉ ARCHIVOS PLANOS?

¿POR QUÉ USAR ARCHIVOS PLANOS EN UN ALMACÉN DE QUÍMICOS?

- **SISTEMAS SIMPLES Y CONFIABLES**
NO DEPENDEN DE SERVIDORES NI BASES DE DATOS COMPLEJAS
- **FUNCIONAMIENTO LOCAL**
IDEALES PARA ENTORNOS INDUSTRIALES SIN CONEXIÓN CONSTANTE
- **RESPUESTA RÁPIDA**
PERMITEN VALIDAR ACCESOS DE FORMA INMEDIATA
- **FÁCIL IMPLEMENTACIÓN**
REDUCEN COSTOS Y COMPLEJIDAD DEL SISTEMA
- **REGISTRO CLARO DE EVENTOS**
EL ARCHIVO TXT PERMITE AUDITORÍA DIRECTA Y ORDENADA



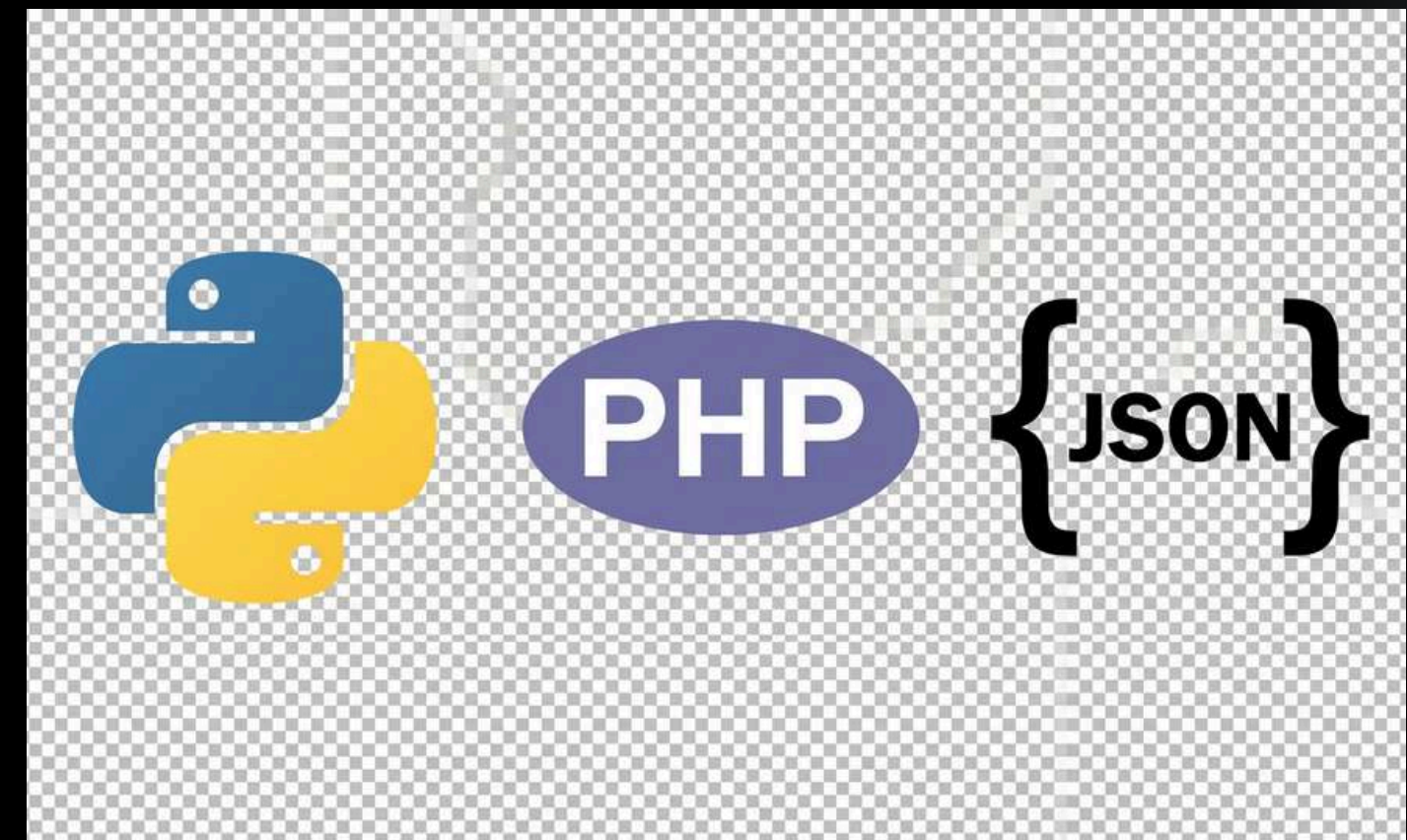
ARQUITECTURA DEL SISTEMA

EL SISTEMA ESTÁ DIVIDIDO EN TRES CAPAS:

1. CONFIGURACIÓN (JSON)
DEFINE LOS USUARIOS Y NIVELES DE ACCESO

2. CONTROL (PYTHON)
PROCESA EL ACCESO Y TOMA DECISIONES

3. VISUALIZACIÓN (PHP)
MUESTRA LOS EVENTOS AL USUARIO



JSON (CONFIGURACIÓN)

TECH COMPANY

ARCHIVO: USUARIOS.JSON

CONTIENE:

- **ID DE TARJETA**
- **NOMBRE DEL EMPLEADO**
- **DEPARTAMENTO**
- **NIVEL DE SEGURIDAD**

FUNCIÓN:

DEFINE QUÉ USUARIOS PUEDEN ACCEDER AL SISTEMA

RELACIÓN CON EL LOG:

- **LOS DATOS DEL JSON SON UTILIZADOS POR PYTHON PARA VALIDAR CADA ACCESO**
- **EL RESULTADO DE ESA VALIDACIÓN SE REGISTRA EN AUDITORIA.TXT**
- **EL LOG REFLEJA SI EL USUARIO FUE AUTORIZADO**
 - **O RECHAZADO SEGÚN LA INFORMACIÓN DEL JSON**



```
{ } usuarios.json > ...
```

```
1  [
2    {
3      "id_tarjeta": "777",
4      "nombre_empleado": "Jahaziel Fraire",
5      "departamento": "Produccion",
6      "nivel_seguridad": 1
7    },
8    {
9      "id_tarjeta": "123",
10     "nombre_empleado": "Jesus Arreola",
11     "departamento": "Admin",
12     "nivel_seguridad": 2
13   },
14   {
15     "id_tarjeta": "456",
16     "nombre_empleado": "Adonai Molina",
17     "departamento": "Admin",
18     "nivel_seguridad": 3
19   }
20 ]
```

PYTHON (CONTROL)

TEXTO:

FUNCIONES PRINCIPALES:

- **CARGAR EL ARCHIVO JSON**
- **BUSCAR EL ID INGRESADO**
- **VALIDAR NIVEL DE SEGURIDAD**
- **SIMULAR APERTURA O ALARMA**
- **REGISTRAR EL EVENTO EN AUDITORÍA**



control_acceso.py > main

```
1  import json
2  from datetime import datetime
3
4  # Nivel mínimo requerido para permitir acceso al almacén
5  # Este valor simula la política de seguridad del sistema
6  NIVEL_REQUERIDO = 2
7
8  # Cargar usuarios desde el archivo JSON
9  def cargar_usuarios():
10     try:
11         # Se abre el archivo en modo lectura ('r') porque solo se necesita consultar datos
12         with open("usuarios.json", "r") as archivo:
13             # json.load convierte el contenido del archivo en una estructura de datos de Python (lista/diccionarios)
14             datos = json.load(archivo)
15             return datos
16     except FileNotFoundError:
17         # Manejo de error si el archivo no existe
18         print("ERROR: No se encontró el archivo usuarios.json")
19         return []
20     except json.JSONDecodeError:
21         # Manejo de error si el JSON está mal estructurado
22         print("ERROR: El archivo JSON está mal formado")
23         return []
```



```
24
25 # Buscar usuario por ID dentro de la lista cargada desde el JSON
26 def buscar_usuario(usuarios, id_ingresado):
27     # Se recorre cada usuario para comparar su ID con el ingresado
28     for usuario in usuarios:
29         if usuario["id_tarjeta"] == id_ingresado:
30             # Si se encuentra coincidencia, se devuelve el usuario completo
31             return usuario
32     # Si no se encuentra, se retorna None (usuario no registrado)
33     return None
34
35 # Registrar el resultado del acceso en el archivo de auditoría
36 def registrar_log(mensaje):
37     # Se usa modo 'a' (append) para NO borrar registros anteriores
38     with open("auditoria.txt", "a") as archivo:
39         # Se obtiene fecha y hora actual para llevar trazabilidad
40         fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
41         # Se guarda el evento en formato estructurado
42         archivo.write(f"{fecha_hora} - {mensaje}\n")
43
44 # Programa principal (simulación del sistema de acceso)
45 def main():
46     # Se cargan los usuarios desde el JSON
47     usuarios = cargar_usuarios()
```

```
48
49 # Si no se cargaron datos, el sistema no continúa
50 if not usuarios:
51     return
52
53 # Ciclo infinito que simula un sistema en funcionamiento continuo
54 while True:
55     print("\n--- CONTROL DE ACCESO ---")
56
57     # Simulación de lector de tarjetas (entrada manual)
58     id_ingresado = input("Ingresa ID de tarjeta (o escribe 'salir'): ")
59
60     # Permite finalizar el sistema manualmente
61     if id_ingresado.lower() == "salir":
62         print("Sistema finalizado.")
63         break
64
65     # Se busca el usuario en el JSON
66     usuario = buscar_usuario(usuarios, id_ingresado)
67
68     # Caso 1: Usuario encontrado
69     if usuario:
70         # Validación del nivel de seguridad requerido
71         if usuario["nivel_seguridad"] >= NIVEL_REQUERIDO:
72             # Simulación de acceso permitido (apertura de puerta)
73             print("CERRADURA ABIERTA por 5 segundos")
74
```

```
75 |         # Mensaje estructurado que se guardará en el log
76 |         mensaje = f"ACCESO PERMITIDO - ID: {id_ingresado} - {usuario['nombre_empleado']} - ACCESO CORRECTO"
77 |     else:
78 |         # Usuario existe pero no tiene permisos suficientes
79 |         print("ACCESO DENEGADO - Nivel de seguridad insuficiente")
80 |
81 |         mensaje = f"ACCESO DENEGADO - ID: {id_ingresado} - {usuario['nombre_empleado']} - NIVEL INSUFICIENTE"
82 |
83 |     # Caso 2: Usuario no registrado
84 | else:
85 |     # Simulación de alarma del sistema
86 |     print("ALARMA ACTIVADA - Intento de acceso no autorizado")
87 |
88 |     mensaje = f"ACCESO DENEGADO - ID: {id_ingresado} - DESCONOCIDO - USUARIO NO REGISTRADO"
89 |
90 |     # Se registra el resultado en el archivo de auditoría
91 |     registrar_log(mensaje)
92 |
93 | # Punto de entrada del programa
94 | if __name__ == "__main__":
95 |     main()
```


PHP (VISUALIZACIÓN)

FUNCIONES PRINCIPALES:

- **LEER EL ARCHIVO AUDITORIA.TXT**
- **MOSTRAR LOS REGISTROS EN UNA TABLA**
- **RESALTAR ACCESOS DENEGADOS**
- **PERMITIR BÚSQUEDA DE EVENTOS**



index.php > html > head > style > @keyframes parpadeo

```
1  <?php
2  $archivo = "auditoria.txt";
3  $lineas = [];
4
5  // Leer archivo
6  if (file_exists($archivo)) {
7      $lineas = file($archivo);
8      $lineas = array_reverse($lineas);
9  }
10
11 // Limpiar historial
12 if (isset($_POST['limpiar'])) {
13     file_put_contents($archivo, "");
14     header("Refresh:0");
15 }
16
17 // Contadores
18 $total = count($lineas);
19 $permitidos = 0;
20 $denegados = 0;
21
22 foreach ($lineas as $linea) {
23     if (strpos($linea, "DENEGADO") !== false) {
24         $denegados++;
25     } else {
26         $permitidos++;
27     }
28 }
29 ?>
30
31 <!DOCTYPE html>
32 <html lang="es">
33 <head>
34 <meta charset="UTF-8">
35 <title>Dashboard Control de Acceso</title>
```

```
66 }
67
68 .stats {
69     display: flex;
70     justify-content: center;
71     gap: 20px;
72     margin-bottom: 20px;
73 }
74
75 .card {
76     padding: 15px;
77     border-radius: 10px;
78     background: #1e293b;
79 }
80
81 .ok { background: #14532d; }
82 .bad { background: #7f1d1d; }
83
84 button {
85     padding: 10px;
86     border-radius: 8px;
87     border: none;
88     cursor: pointer;
89     background: #334155;
90     color: white;
91 }
92
93 button:hover {
94     background: #475569;
95 }
96
97 input {
98     padding: 10px;
99     border-radius: 8px;
100    border: none;
101 }
102
```

```

127         fila.style.display = texto.includes(input) ? "" : "none";
128     });
129 }
130 </script>
131 </head>
132
133 <body>
134
135 <header>CONTROL DE ACCESOS</header>
136
137 <div class="container">
138
139 <div class="stats">
140     <div class="card">Total: <?php echo $total; ?></div>
141     <div class="card ok">Permitidos: <?php echo $permitidos; ?></div>
142     <div class="card bad">Denegados: <?php echo $denegados; ?></div>
143 </div>
144
145 <form method="POST" style="text-align:center;">
146 |     <button name="limpiar">Limpiar historial</button>
147 </form>
148
149 <div style="text-align:center; margin:10px;">
150 |     <input type="text" id="buscador" onkeyup="filtrar()" placeholder="Buscar por ID o nombre...">
151 </div>
152
153 <table>
154 <thead>
155 <tr>
156 <th>Fecha</th>
157 <th>Estado</th>
158 <th>ID</th>
159 <th>Usuario</th>
160 <th>Detalle</th>
161 </tr>
162 </thead>
163
164 <tbody>
165 <?php foreach ($lineas as $index => $linea):
166
167     $partes = explode(" - ", $linea);
168
169     $fecha = $partes[0] ?? "";
170     $estado = $partes[1] ?? "";
171     $id = str_replace("ID: ", "", $partes[2] ?? "");
172     $nombre = $partes[3] ?? "";
173     $detalle = $partes[4] ?? "";
174
175     // Solo por seguridad visual mínima
176     if (empty(trim($nombre))) {
177         $nombre = "Desconocido";
178     }
179
180     $clase = (strpos($linea, "DENEGADO") !== false) ? "denegado" : "permitido";
181     ?>
182
183 <tr class="<?php echo $clase; ?> <?php echo $index == 0 ? 'ultimo' : ''; ?> <?php echo ($index == 0 && $clase == 'denegado') ? 'alerta' : ''; ?>">
184 <td><?php echo $fecha; ?></td>
185 <td><?php echo $estado; ?></td>
186 <td><?php echo $id; ?></td>

```

```

187 <td><?php echo $nombre; ?></td>
188 <td><?php echo $detalle; ?></td>
189 </tr>
190
191 <?php endforeach; ?>
192 </tbody>
193 </table>
194
195 </div>
196
197 </body>
198 </html>

```


FLUJO DEL SISTEMA



DEMOSTRACIÓN

PRUEBAS DEL SISTEMA:

- ACCESO VÁLIDO
- ACCESO DENEGADO
- USUARIO NO REGISTRADO
- ACTUALIZACIÓN EN TIEMPO REAL



EJEMPLO DE LOG

EJEMPLO DE AUDITORÍA:

≡ auditoria.txt

```
1  2026-04-19 17:49:34 - ACCESO PERMITIDO - ID: 123 - Jesus Arreola - ACCESO CORRECTO
2  2026-04-19 17:49:37 - ACCESO PERMITIDO - ID: 456 - Adonai Molina - ACCESO CORRECTO
3  2026-04-19 17:49:38 - ACCESO DENEGADO - ID: 777 - Jahaziel Fraire - NIVEL INSUFICIENTE
4  2026-04-19 17:49:46 - ACCESO DENEGADO - ID: 888 - DESCONOCIDO - USUARIO NO REGISTRADO
```

CADA EVENTO INCLUYE FECHA, HORA Y RESULTADO

DECISIONES TÉCNICAS

MANEJO DE ARCHIVOS EN EL SISTEMA:

MODO 'R' (READ):

- SE UTILIZA PARA LEER EL ARCHIVO USUARIOS.JSON
- PERMITE ACCEDER A LOS DATOS SIN MODIFICARLOS

MODO 'A' (APPEND):

- SE UTILIZA PARA ESCRIBIR EN AUDITORIA.TXT
- AGREGA NUEVA INFORMACIÓN SIN BORRAR REGISTROS ANTERIORES

VENTAJAS DEL ENFOQUE:

- SE CONSERVA EL HISTORIAL COMPLETO DE ACCESOS
- SE EVITA LA PÉRDIDA DE INFORMACIÓN
- SE MANTIENE UN REGISTRO CONTINUO Y ORDENADO
- PERMITE AUDITORÍA Y TRAZABILIDAD DEL SISTEMA

MANEJO DE ERRORES

MANEJO DE ERRORES EN EL SISTEMA:

- **ARCHIVO JSON INEXISTENTE**
EL SISTEMA DETECTA SI NO SE ENCUENTRA EL ARCHIVO USUARIOS.JSON
Y EVITA QUE EL PROGRAMA FALLE
- **JSON MAL ESTRUCTURADO**
SE VALIDA EL FORMATO PARA PREVENIR ERRORES AL CARGAR LOS DATOS
- **ID NO REGISTRADO**
SE DETECTAN ACCESOS INVÁLIDOS Y SE ACTIVA UNA ALERTA

IMPORTANCIA:

- **EVITA FALLOS CRÍTICOS DEL SISTEMA**
- **PERMITE FUNCIONAMIENTO CONTINUO**
- **AUMENTA LA CONFIABILIDAD EN ENTORNOS INDUSTRIALES**

MAQUETA DIGITAL – LECTOR DE TARJETAS FÍSICO

Entrada de la Fábrica

PANTALLA LCD

Muestra mensajes e información del sistema.

LECTOR RFID

Zona de lectura para tarjetas de acceso.

INDICADORES LED

Rojo: Acceso denegado
Verde: Acceso permitido

BUZZER (ALARMA)

Se activa en caso de acceso no autorizado.



CERRADURA

Se activa para permitir el acceso durante 5 segundos si el usuario es válido.

CARACTERÍSTICAS



Validación de usuario

Se verifica el ID en el archivo JSON (Python).



Control de acceso

Nivel de seguridad mínimo requerido: 2



Registro de eventos

Cada intento se guarda en auditoria.txt con fecha y hora (LOG).



Monitoreo web

Visualización en tiempo real en el dashboard (PHP).

FLUJO DEL SISTEMA



Este sistema permite controlar y monitorear los accesos a la fábrica, garantizando seguridad, trazabilidad y control en tiempo real.

TECNOLOGÍAS UTILIZADAS

Python • JSON • TXT (LOG) • PHP • HTML • CSS • JavaScript

CONCLUSIÓN

EL SISTEMA DESARROLLADO PERMITE CONTROLAR DE MANERA SEGURA EL ACCESO A UN ALMACÉN DE PRODUCTOS QUÍMICOS MEDIANTE UNA ARQUITECTURA BASADA EN ARCHIVOS PLANOS. A TRAVÉS DEL USO DE JSON PARA LA CONFIGURACIÓN DE USUARIOS, PYTHON PARA EL CONTROL DE ACCESO Y PHP PARA LA VISUALIZACIÓN, SE LOGRA UNA SOLUCIÓN MODULAR, CONFIABLE Y EFICIENTE. ADEMÁS, EL REGISTRO CONTINUO DE EVENTOS EN ARCHIVOS DE AUDITORÍA GARANTIZA LA TRAZABILIDAD DE CADA ACCESO, HACIENDO QUE EL SISTEMA SEA APLICABLE EN ENTORNOS INDUSTRIALES DONDE LA SEGURIDAD Y EL CONTROL SON FUNDAMENTALES.



THANK YOU